

UNIVERSITY OF FRONTIER TECHNOLOGY, BANGLADESH

DEPARTMENT OF INTERNET OF THINGS AND ROBOTICS ENGINEERING (IRE)

FACULTY OF CYBER PHYSICAL SYSTEM ENGINEERING

LAB REPORT 03

DART COLLECTIONS, FILE HANDLING AND CLI TO-DO APPLICATION

(List, Set, Map, File I/O, Higher-Order Functions, and CLI Architecture)

COURSE CODE: IRE 460

COURSE TITLE: MOBILE PLATFORM FOR IOT DEVICES SESSIONAL

SUBMITTED BY:

Shahriar Alom Masud

Student ID: **2201032** | Session: 2022–2023

Department of Internet of Things and Robotics Engineering (IRE), UFTB

SUBMITTED TO:

Md. Rafiqul Islam

Lecturer

Department of Internet of Things and Robotics Engineering (IRE)

University of Frontier Technology, Bangladesh

NOVEMBER 03, 2026

Laboratory Objectives and Problem Overview

This laboratory module focuses on fundamental data structures and input/output systems in the Dart programming language, essential for building mobile applications on Flutter. The tasks cover:

1. **Problem 1:** Create a list of names and print all names using list.
2. **Problem 2:** Create a set of fruits and print all fruits using loop.
3. **Problem 3:** Create a program that reads list of expenses amount using user input and print total.
4. **Problem 4:** Create an empty list of type string called `days`. Use the `add` method to add names of 7 days and print all days.
5. **Problem 5:** Read data from a file, capitalize each word, and write the result into a separate file in the same format.
6. **Problem 6:** Create a map with `name`, `address`, `age`, `country` keys and store values to it. Update country name to another country and print all keys and values.
7. **Problem 7:** Create a map with contact names and phone keys, and use `where` to find all keys that have length 4.
8. **Problem 8:** Create a simple to-do application that allows the user to add, remove, and view their tasks.

GitHub Repository Link:
https://github.com/Masud744/flutter-lab/tree/main/Lab_03

Problem 1: List of Names and Printing

Task Requirement: Create a list of names and print all names using list.
Implementation Details: An indexed collection of type `List<String>` is populated with names. Dart offers direct formatting as well as standard iteration.

Source Code (`src/problem1_names_list.dart`)

```dart
// Lab 03 - Problem 1: Create a list of names and print all names using list.

void main() {
  // Creating a list of string names
  List<String> names = ['Shahriar', 'Masud', 'Rafiqul', 'Tanvir', 'Sakib'];

  print('========================================');
  print('         Problem 1: List of Names       ');
  print('========================================');

  // Printing all names directly using list
  print('\nPrinting entire list directly:');
  print(names);

  // Printing names by iterating through the list
  print('\nPrinting names individually using loop:');
  for (int i = 0; i < names.length; i++) {
    print('${i + 1}. ${names[i]}');
  }
}
```

Program Execution Output

```
========================================
        Problem 1: List of Names
========================================

Printing entire list directly:
[Shahriar, Masud, Rafiqul, Tanvir, Sakib]

Printing names individually using loop:
1. Shahriar
2. Masud
3. Rafiqul
4. Tanvir
5. Sakib
```

Problem 2: Set of Fruits and Loop Iteration

Task Requirement: Create a set of fruits and print all fruits using loop.
Implementation Details: A `Set<String>` is an unordered collection of unique elements. The elements are iterated over using an enhanced `for-in` loop.

Source Code (`src/problem2_fruits_set.dart`)

```dart
// Lab 03 - Problem 2: Create a set of fruits and print all fruits using loop.

void main() {
  // Creating a Set of unique fruits
  Set<String> fruits = {'Apple', 'Banana', 'Mango', 'Orange', 'Strawberry', 'Pineapple'
      };

  print('========================================');
  print('         Problem 2: Set of Fruits        ');
  print('========================================');

  print('\nPrinting all fruits using for-in loop:');
  int index = 1;
  for (String fruit in fruits) {
    print('$index. $fruit');
    index++;
  }

  print('\nTotal unique fruits count: ${fruits.length}');
}
```

Program Execution Output

```
========================================
        Problem 2: Set of Fruits
========================================

Printing all fruits using for-in loop:
1. Apple
2. Banana
3. Mango
4. Orange
5. Strawberry
6. Pineapple

Total unique fruits count: 6
```

Problem 3: User Input Expenses Calculator

Task Requirement: Create a program that reads list of expenses amount using user input and print total.
Implementation Details: Utilizing `dart:io`, the application prompts the user for the number of expenses and their respective numeric amounts, storing them in a `List<double>`. The cumulative sum is computed cleanly using the `.fold()` method.

Source Code (`src/problem3_expenses.dart`)

```dart
// Lab 03 - Problem 3: Create a program that reads list of expenses amount using user
    input and print total.

import 'dart:io';
```

```
4
5  void main() {
6    List<double> expenses = [];
7
8    print('========================================');
9    print('     Problem 3: Expenses Calculator     ');
10   print('========================================');
11
12   stdout.write('Enter the number of expenses you want to input: ');
13   String? countInput = stdin.readLineSync();
14   int count = int.tryParse(countInput ?? '') ?? 0;
15
16   if (count <= 0) {
17     print('No expenses entered or invalid count.');
18     return;
19   }
20
21   for (int i = 1; i <= count; i++) {
22     stdout.write('Enter expense #$i amount: ');
23     String? amountInput = stdin.readLineSync();
24     double amount = double.tryParse(amountInput ?? '') ?? 0.0;
25     expenses.add(amount);
26   }
27
28   // Calculating total expense using fold / reduce
29   double totalExpense = expenses.fold(0.0, (previousValue, element) => previousValue +
         element);
30
31   print('\n---------------- SUMMARY ----------------');
32   print('List of entered expenses: $expenses');
33   print('Total number of items: ${expenses.length}');
34   print('Total Expenses: \$${totalExpense.toStringAsFixed(2)}');
35   print('-----------------------------------------');
36 }
```

Program Execution Output

```
========================================
     Problem 3: Expenses Calculator
========================================
Enter the number of expenses you want to input: 3
Enter expense #1 amount: 120.50
Enter expense #2 amount: 45.00
Enter expense #3 amount: 84.50

---------------- SUMMARY ----------------
List of entered expenses: [120.5, 45.0, 84.5]
Total number of items: 3
Total Expenses: $250.00
-----------------------------------------
```

Problem 4: Days of the Week Dynamic List

Task Requirement: Create an empty list of type string called `days`. Use the `add` method to add names of 7 days and print all days.
Implementation Details: A dynamic growable list `List<String> days = [];` is initialized empty and sequentially populated using seven distinct `.add()` method calls.

Source Code (`src/problem4_days_list.dart`)

```dart
// Lab 03 - Problem 4: Create an empty list of type string called days.
// Use the add method to add names of 7 days and print all days.

void main() {
  print('========================================');
  print('    Problem 4: Days of the Week List    ');
  print('========================================');

  // Step 1: Create an empty list of type String called days
  List<String> days = [];
  print('Initial list: $days (Length: ${days.length})');

  // Step 2: Use the add method to add names of 7 days
  days.add('Monday');
  days.add('Tuesday');
  days.add('Wednesday');
  days.add('Thursday');
  days.add('Friday');
  days.add('Saturday');
  days.add('Sunday');

  // Step 3: Print all days
  print('\nPrinting all days after using add() method:');
  for (int i = 0; i < days.length; i++) {
    print('Day ${i + 1}: ${days[i]}');
  }

  print('\nComplete list representation: $days');
}
```

Program Execution Output

```
========================================
    Problem 4: Days of the Week List
========================================
Initial list: [] (Length: 0)

Printing all days after using add() method:
Day 1: Monday
Day 2: Tuesday
Day 3: Wednesday
Day 4: Thursday
Day 5: Friday
Day 6: Saturday
Day 7: Sunday

Complete list representation: [Monday, Tuesday, Wednesday, Thursday, Friday, Saturday, Sunday]
```

Problem 5: File Word Capitalization

Task Requirement: Read data from a file and capitalized each word and write the result in separate file. Both file should be in same format.

Implementation Details: Utilizing the `File` class from `dart:io`, the text from `input.txt` is read synchronously, transformed such that each token begins with an uppercase letter followed by lowercase characters, and written to `output.txt`.

Source Code (src/problem5_file_capitalize.dart)

```dart
// Lab 03 - Problem 5: Read data from a file and capitalized each word and write the
    result in separate file.
// Both file should be in same format.

import 'dart:io';

String capitalizeWords(String text) {
  // Splits line into words, capitalizes first character of each word
  return text.split(' ').map((word) {
    if (word.isEmpty) return word;
    return word[0].toUpperCase() + word.substring(1).toLowerCase();
  }).join(' ');
}

void main() {
  print('========================================');
  print(' Problem 5: File Word Capitalization   ');
  print('========================================');

  // Resolve directory of the current script to allow running from any folder
  final scriptDir = File(Platform.script.toFilePath()).parent.path;
  final inputFile = File('$scriptDir/input.txt');
  final outputFile = File('$scriptDir/output.txt');

  if (!inputFile.existsSync()) {
    inputFile.writeAsStringSync(
      'mobile platform for iot devices sessional\n'
      'learning flutter and dart collection libraries\n'
      'file input output operations in dart programming\n',
    );
  }

  // Reading original content
  String content = inputFile.readAsStringSync();
  print('\n[1] Original content from "input.txt":');
  print('----------------------------------------');
  print(content.trim());
  print('----------------------------------------');

  // Processing and capitalizing each word per line
  List<String> lines = content.split('\n');
  List<String> processedLines = lines.map((line) => capitalizeWords(line)).toList();
  String capitalizedResult = processedLines.join('\n');

  // Writing output to separate file in the same (.txt) format
  outputFile.writeAsStringSync(capitalizedResult);

  // Reading and displaying the result
  print('\n[2] Processed content written to "output.txt":');
  print('----------------------------------------');
  print(outputFile.readAsStringSync().trim());
  print('----------------------------------------');

  print('\n[SUCCESS] Successfully processed file and saved capitalized data to output.
      txt');
}
```

Program Execution Output

```
========================================
 Problem 5: File Word Capitalization
========================================
```

```
[1] Original content from "input.txt":
----------------------------------------
mobile platform for iot devices sessional
learning flutter and dart collection libraries
file input output operations in dart programming
----------------------------------------

[2] Processed content written to "output.txt":
----------------------------------------
Mobile Platform For Iot Devices Sessional
Learning Flutter And Dart Collection Libraries
File Input Output Operations In Dart Programming
----------------------------------------

[SUCCESS] Successfully processed file and saved capitalized data to output.txt
```

Problem 6: Map Data Manipulation & Key Update

Task Requirement: Create a map with `name`, `address`, `age`, `country` keys and store values to it. Update country name to other country and print all keys and values.
Implementation Details: Demonstrates associative key-value mapping using `Map<String, dynamic>`, index-based value modification, and iteration over key-value entries.

Source Code (src/problem6_person_map.dart)

```dart
// Lab 03 - Problem 6: Create a map with name, address, age, country keys and store values to it.
// Update country name to other country and print all keys and values.

void main() {
  print('========================================');
  print('   Problem 6: Person Map & Update Key   ');
  print('========================================');

  // Step 1: Create a map with name, address, age, country keys
  Map<String, dynamic> person = {
    'name': 'Shahriar Alom Masud',
    'address': 'Gazipur, Dhaka',
    'age': 22,
    'country': 'Bangladesh',
  };

  print('\n--- Initial Map Data ---');
  person.forEach((key, value) {
    print('$key : $value');
  });

  // Step 2: Update country name to other country
  print('\nUpdating country: "${person['country']}" -> "Japan"...');
  person['country'] = 'Japan';

  // Step 3: Print all keys and values
  print('\n--- Updated Map Data (All Keys and Values) ---');
  for (var entry in person.entries) {
    print('${entry.key.padRight(10)}: ${entry.value}');
  }
}
```

Program Execution Output

```
========================================
   Problem 6: Person Map & Update Key
========================================

--- Initial Map Data ---
name : Shahriar Alom Masud
address : Gazipur, Dhaka
age : 22
country : Bangladesh

Updating country: "Bangladesh" -> "Japan"...

--- Updated Map Data (All Keys and Values) ---
name      : Shahriar Alom Masud
address   : Gazipur, Dhaka
age       : 22
country   : Japan
```

Problem 7: Map Filtering with `where` Method

Task Requirement: Create a map with name, phone keys and store some values to it. Use `where` to find all keys that have length 4.
Implementation Details: Demonstrates higher-order functional querying on map keys (`contacts.keys.where((k => k.length == 4)`).

Source Code (`src/problem7_phone_map.dart`)

```dart
// Lab 03 - Problem 7: Create a map with name, phone keys and store some values to it.
// Use where to find all keys that have length 4.

void main() {
  print('========================================');
  print('   Problem 7: Map Filtering with where  ');
  print('========================================');

  // Creating a Map where keys represent contact names and values represent phone numbers
  Map<String, String> contacts = {
    'John': '01711001122',
    'Alex': '01822334455',
    'Masud': '01933445566',
    'Sara': '01544556677',
    'Rifat': '01655667788',
    'Liam': '01366778899',
    'Tanvir': '01777889900',
  };

  print('\n--- All Contacts in Map ---');
  contacts.forEach((name, phone) {
    print('${name.padRight(8)} (length: ${name.length}) -> $phone');
  });

  // Using .where() on keys to find all keys with length 4
  Iterable<String> length4Keys = contacts.keys.where((key) => key.length == 4);

  print('\n--- Keys Having Length 4 (Filtered via .where()) ---');
  for (String key in length4Keys) {
    print('Key: "$key" | Phone: ${contacts[key]}');
  }

  print('\nFiltered Keys List: ${length4Keys.toList()}');
}
```

Program Execution Output

```
========================================
   Problem 7: Map Filtering with where
========================================

--- All Contacts in Map ---
John     (length: 4) -> 01711001122
Alex     (length: 4) -> 01822334455
Masud    (length: 5) -> 01933445566
Sara     (length: 4) -> 01544556677
Rifat    (length: 5) -> 01655667788
Liam     (length: 4) -> 01366778899
Tanvir   (length: 6) -> 01777889900

--- Keys Having Length 4 (Filtered via .where()) ---
Key: "John" | Phone: 01711001122
Key: "Alex" | Phone: 01822334455
Key: "Sara" | Phone: 01544556677
Key: "Liam" | Phone: 01366778899

Filtered Keys List: [John, Alex, Sara, Liam]
```

Problem 8: Interactive CLI To-Do Application

Task Requirement: Create a simple to-do application that allows user to add, remove, and view their task.
Implementation Details: Built with a modular `TodoApp` class encapsulating a private list of task strings, accompanied by clean CRUD methods: `addTask()`, `removeTask()`, and `viewTasks()`. Includes an interactive menu loop with input validation.

Source Code (`src/problem8_todo_app.dart`)

```dart
// Lab 03 - Problem 8: Create a simple to-do application that allows user to add, remove, and view their task.

import 'dart:io';

class TodoApp {
  final List<String> _tasks = [];

  // Method to add a new task
  void addTask(String task) {
    String trimmed = task.trim();
    if (trimmed.isNotEmpty) {
      _tasks.add(trimmed);
      print('[+] Success: Task "$trimmed" added to list.');
    } else {
      print('[-] Error: Task description cannot be empty.');
    }
  }

  // Method to remove a task by its 1-based index
  bool removeTask(int taskNumber) {
    if (taskNumber >= 1 && taskNumber <= _tasks.length) {
      String removed = _tasks.removeAt(taskNumber - 1);
      print('[+] Success: Task #$taskNumber ("$removed") has been removed.');
      return true;
    } else {
      print('[-] Error: Invalid task number. Please enter a number between 1 and ${_tasks.length}.');
      return false;
    }
  }

  // Method to view all active tasks
  void viewTasks() {
    print('\n================ CURRENT TO-DO LIST ================');
    if (_tasks.isEmpty) {
      print('  (Your to-do list is empty! Add tasks to get started)');
    } else {
      for (int i = 0; i < _tasks.length; i++) {
        print('  [${i + 1}] ${_tasks[i]}');
      }
    }
    print('====================================================\n');
  }

  // Method to run automated demo for testing
  void runDemo() {
    print('========================================');
    print('   To-Do Application Demonstration      ');
    print('========================================');

    print('\n--- 1. Viewing Initial Empty List ---');
    viewTasks();

    print('--- 2. Adding Tasks ---');
    addTask('Complete IRE 460 Lab Report 03');
    addTask('Study Flutter Widget Lifecycle');
    addTask('Prepare IoT Project Presentation');
```

```
57
58      print('\n--- 3. Viewing List After Additions ---');
59      viewTasks();
60
61      print('--- 4. Removing Task #2 ("Study Flutter Widget Lifecycle") ---');
62      removeTask(2);
63
64      print('\n--- 5. Viewing Final List ---');
65      viewTasks();
66    }
67
68    // Interactive CLI loop
69    void runInteractive() {
70      print('========================================');
71      print('      CLI TO-DO APPLICATION SYSTEM      ');
72      print('========================================');
73
74      while (true) {
75        print('\nChoose an action:');
76        print('1. View Tasks');
77        print('2. Add Task');
78        print('3. Remove Task');
79        print('4. Exit');
80        stdout.write('Enter your choice (1-4): ');
81
82        String? choice = stdin.readLineSync()?.trim();
83
84        switch (choice) {
85          case '1':
86            viewTasks();
87            break;
88          case '2':
89            stdout.write('Enter new task description: ');
90            String? task = stdin.readLineSync();
91            if (task != null) addTask(task);
92            break;
93          case '3':
94            viewTasks();
95            if (_tasks.isNotEmpty) {
96              stdout.write('Enter task number to remove: ');
97              int? num = int.tryParse(stdin.readLineSync() ?? '');
98              if (num != null) {
99                removeTask(num);
100             } else {
101               print('[-] Error: Please enter a valid numerical index.');
102             }
103           }
104           break;
105         case '4':
106           print('\nExiting To-Do Application. Have a productive day!');
107           return;
108         default:
109           print('[-] Invalid selection. Please choose an option from 1 to 4.');
110       }
111     }
112   }
113 }
114
115 void main(List<String> args) {
116   final app = TodoApp();
117   if (args.contains('--demo')) {
118     app.runDemo();
119   } else {
120     app.runInteractive();
121   }
122 }
```

Program Execution Output

```
========================================
   To-Do Application Demonstration
```

```
========================================

--- 1. Viewing Initial Empty List ---

================ CURRENT TO-DO LIST ================
  (Your to-do list is empty! Add tasks to get started)
====================================================

--- 2. Adding Tasks ---
[+] Success: Task "Complete IRE 460 Lab Report 03" added to list.
[+] Success: Task "Study Flutter Widget Lifecycle" added to list.
[+] Success: Task "Prepare IoT Project Presentation" added to list.

--- 3. Viewing List After Additions ---

================ CURRENT TO-DO LIST ================
  [1] Complete IRE 460 Lab Report 03
  [2] Study Flutter Widget Lifecycle
  [3] Prepare IoT Project Presentation
====================================================

--- 4. Removing Task #2 ("Study Flutter Widget Lifecycle") ---
[+] Success: Task #2 ("Study Flutter Widget Lifecycle") has been removed.

--- 5. Viewing Final List ---

================ CURRENT TO-DO LIST ================
  [1] Complete IRE 460 Lab Report 03
  [2] Prepare IoT Project Presentation
====================================================
```

Conclusion

In this laboratory session, all eight problem statements covering fundamental and intermediate Dart constructs—including dynamic lists, unique sets, associative maps, predicate filtering using `.where()`, standard file input/output, and console menu-driven application design—were successfully implemented, verified, and documented.